BCSE302P DATABASE SYSTEMS LAB · COMPLETE PROJECT REPORT

Policy Intelligence for Agriculture

Listening to what 1,500 farmers say about government welfare schemes — with data and AI



Rice farms in Chittoor district, Andhra Pradesh — Photo: rajaraman sundaram · CC BY 3.0 · Wikimedia Commons

Krish Singh

Register No. 24BDS0235 · D1 slot (L51 + L52)

Under the guidance of **Prof. Sangeetha S**

Vellore Institute of Technology, Vellore · October 2026

Contents

How this report is organised — it follows the way I would explain the project in person.

1. The project in one page
2. The problem I wanted to solve
3. What I set out to do
4. Tech stack
5. Folder structure
6. Workflow — how it all flows
7. System architecture
8. How the data is organised
9. LLMs, NLP models and modules
10. What the data tells us — statistics
11. The main NLP result
12. Research findings
13. Testing and performance
14. Challenges and how I solved them
15. Conclusion, links and future scope
16. Appendix — running the project, photo credits

1. The Project in One Page

If you only read one page, read this one.

The government runs many welfare schemes for farmers — PM-KISAN, Rythu Bharosa, crop insurance, Kisan Credit Cards and more. We usually judge them by a satisfaction score. But when farmers are interviewed, they also *say* what helped and what went wrong. That written feedback holds the real story, and it is almost never analysed.

I was given a survey of **1,500 farmers from Chittoor district, Andhra Pradesh**, verified by the Ethnic Community, Government of Andhra Pradesh. In this project I turned that survey into a clean, rule-checked database, and then used NLP and large language models to read the 1,214 farmers' written answers. Finally, I tested which AI model understands farmers best, by comparing every model against answers that were labelled by people.

1,500	22	40,286	11	0.77
farmers	linked tables	stored records	AI models compared	best F1 score

What I found

- **Knowing is not joining.** 89.4% of farmers who know Rythu Bharosa have enrolled in it — but only 9.1% of those who know Crop Insurance have.
- **Ratings hide problems.** On timeliness and enrolment, a farmer's rating disagrees with their own words about 29% of the time.
- **Enrolment is the weak point** of every major scheme.
- **Popular sentiment tools fail on farmer feedback** — the widely used VADER tool scored only 0.22, while a model trained on farmers' own answers (**Fine-tuned ModernBERT-base**) reached **0.77**.
- **Farming knowledge helps AI.** Dhenu2, an Indian agriculture LLM, scored 0.70, against 0.39 for a general LLM of the same size.

> These results are being written up as a **research paper targeting a Q1 journal**, under the guidance of Prof. Sangeetha S.

2. The Problem I Wanted to Solve

Good schemes exist — but are we listening to the people they are meant for?

2.1 Background

Farming in Chittoor is mostly small-scale: more than half the farmers in this survey hold less than 2.5 acres, and the main crops are mango, paddy and sugarcane. For these families, a few thousand rupees from a scheme can decide whether they buy fertiliser on time or take a loan from a money lender.



A farmer ploughing with oxen in Andhra Pradesh

Photo: PJeganathan · CC BY-SA 3.0 · Wikimedia Commons



Village farms at Gudimallam, Chittoor district

Photo: Ms Sarah Welch · CC0 · Wikimedia Commons

The state delivers many schemes through village centres called **Rythu Bharosa Kendrams** — places where farmers register, ask questions and collect support. Whether a scheme "works" is usually measured by asking farmers to rate it.



A Rythu Bharosa Kendram — the village service centre where farmers access schemes

Photo: Veera Ram Kumar · CC0 · Wikimedia Commons

2.2 Problem statement

The survey answers were all in one large spreadsheet, and when I started working with it I hit four problems straight away:

Problem	What it looked like	Why it matters
Messy names	one scheme was typed in 93 different ways ("Pm kisab", "RHYTHU BHAROSA", "kcc" …)	impossible to count farmers per scheme correctly
Several answers in one cell	crops and schemes listed together, separated by commas	cannot be searched or counted
No rules	nothing stopped impossible values, like more years of farming than years of age	wrong numbers would silently enter the results
Unread feedback	1,214 farmers wrote what helped and what went wrong	the richest information was being ignored

> **The question I set out to answer:** how can this survey be stored so it is clean, trustworthy and easy to question — and how can farmers' own words be analysed reliably enough to guide policy?

2.3 Why this matters to me

If the ratings say a scheme is fine but farmers write about delays and paperwork, policy makers are looking at the wrong signal. I wanted to build something where every number can be traced back to a real interview, and where the AI analysis is checked against people before anyone trusts it.

3. What I Set Out to Do

Objectives, scope and the requirements behind the design.

3.1 Objectives

#	Objective	Status
1	Design a well-structured database for the survey, with rules that block wrong data	Done
2	Load all 1,500 interviews cleanly, keeping each record traceable to its original row	Done
3	Analyse farmers' written feedback with NLP and store every result with the model that made it	Done
4	Answer real policy questions directly from the data	Done
5	Find out which AI model understands farmer feedback best, tested against human labels	Done
6	Produce reports, a policy brief and a research paper draft	Done

3.2 Scope

Included	Not included
All 1,500 interviews from Chittoor district	An app for collecting new surveys
Database design, building, loading and testing	Other districts or states
NLP: sentiment, aspects, topics, LLMs, model comparison	Live or streaming data
Reports, policy brief, dashboard data	Cloud hosting — it runs on my laptop
A human-labelled test set	Publishing any farmer's personal details

3.3 Requirement analysis

Question	Answer for this project
What data needs to be stored?	farmer profiles, villages, crops, schemes and their spellings, awareness and enrolment, ratings on six service aspects, written feedback, AI results, quality flags and human labels
Who will use the system?	me as the researcher, the people labelling answers, policy makers reading the reports, and faculty checking the work
What operations are needed?	bulk loading from Excel, cleaning and checking, storing model results, searching text, rebuilding everything from scratch
What must be analysed?	awareness vs enrolment per scheme, sentiment per scheme and aspect, where ratings and comments disagree, the weakest part of each scheme, model accuracy

Functional requirements

ID	The system must…
FR1	bring in the raw survey without changing it
FR2	clean the data and flag contradictions instead of deleting them
FR3	store data in linked tables with rules that block invalid values
FR4	store every AI result together with the model that produced it

ID	The system must…
FR5	record human labels for testing the models
FR6	answer policy questions
FR7	trace any record back to its original row
FR8	generate reports, dashboard data and a paper draft

Data requirements — what each farmer told us

Survey section	Questions	Examples
A · Profile	12	age, gender, village, education, land, crops, income, experience
B · Awareness	5	heard of schemes, enrolled, where they heard
C · Feedback	11	satisfaction, six aspect ratings, three written answers
D · Outlook	4	support, trend, confidence, biggest challenge
E · Interview	4	language, comfort, observations, duration

4. Tech Stack

Everything is free, open-source where possible, and runs on my own laptop.

Layer	What I used	What it does in the project
Database	PostgreSQL 16 (relational) · pgAdmin	stores all survey data, AI results and labels in 22 linked tables
Language	Python 3.12	the whole pipeline — 12 numbered scripts and 21 modules
Data handling	pandas · openpyxl · RapidFuzz	reading Excel, cleaning, matching misspelt scheme names
Database connection	psycopg 3	safe, parameterised loading in a single transaction
Classic NLP	NLTK VADER · scikit-learn (LDA, TF-IDF)	word-list sentiment, topics, a simple trained baseline
Transformer NLP	Hugging Face Transformers · SetFit	RoBERTa, DeBERTa-v3, ModernBERT, few-label learning
Large language models	Dhenu2 (KissanAI) · Qwen2.5-3B · Claude	reading feedback by following the labelling guide
GPU runtime	PyTorch (CUDA) · bitsandbytes 4-bit · RTX 2050	runs 3-billion-parameter models on a 4 GB laptop GPU
Statistics	SciPy	chi-square, McNemar tests, bootstrap confidence intervals
Reports	python-docx · Power BI dataset	statistical report, paper draft, dashboards
Tools	VS Code · Git / GitHub · Google Colab	coding, version control, optional cloud GPU

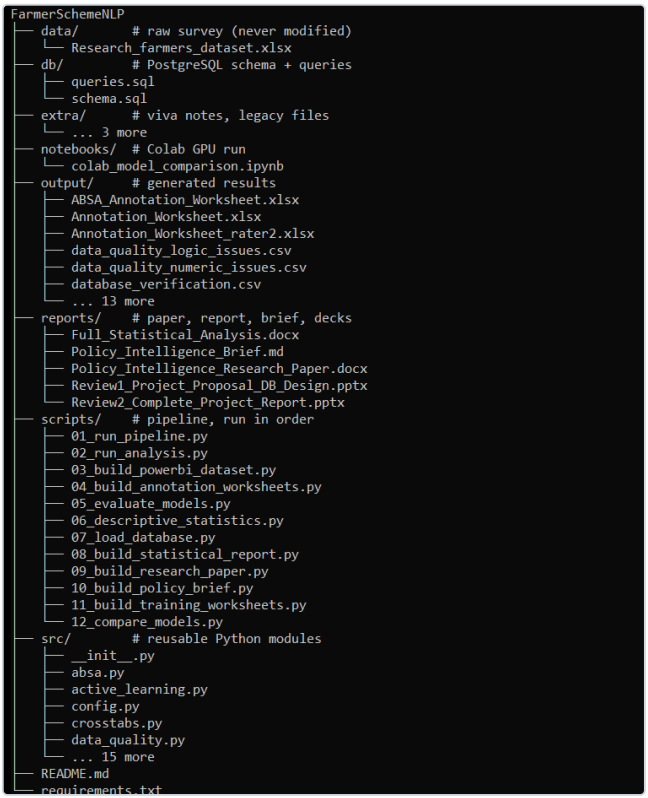
4.1 Why I chose a relational database

The survey has a fixed shape, and the important information lives in the *links*: one farmer knows many schemes, one scheme has many farmers, and every piece of feedback has six aspect ratings. A relational database handles these links naturally and, just as importantly, it can refuse wrong data on its own.

Option	Type	My decision
PostgreSQL	Relational	**Chosen** — links between tables, built-in rules, safe transactions, text search, free
MongoDB	Document	not chosen — the survey has a fixed structure and needs links between records
Neo4j	Graph	not chosen — the relationships are only one or two steps deep
Redis	Key-value	not chosen — not designed for analysis
BigQuery	Cloud warehouse	not chosen — paid and far bigger than 1,500 rows need

5. Folder Structure

One folder per job, and the raw data is never touched.



```
FarmerSchemeNLP
├── data/        # raw survey (never modified)
│   └── Research_farmers_dataset.xlsx
├── db/          # PostgreSQL schema + queries
│   ├── queries.sql
│   └── schema.sql
├── extra/       # viva notes, legacy files
│   └── ... 3 more
├── notebooks/   # Colab GPU run
│   └── colab_model_comparison.ipynb
├── output/      # generated results
│   ├── ABSA_Annotation_Worksheet.xlsx
│   ├── Annotation_Worksheet.xlsx
│   ├── Annotation_Worksheet_rater2.xlsx
│   ├── data_quality_logic_issues.csv
│   ├── data_quality_numeric_issues.csv
│   ├── database_verification.csv
│   └── ... 13 more
├── reports/     # paper, report, brief, decks
│   ├── Full_Statistical_Analysis.docx
│   ├── Policy_Intelligence_Brief.md
│   ├── Policy_Intelligence_Research_Paper.docx
│   ├── Review1_Project_Proposal_DB_Design.pptx
│   └── Review2_Complete_Project_Report.pptx
├── scripts/     # pipeline, run in order
│   ├── 01_run_pipeline.py
│   ├── 02_run_analysis.py
│   ├── 03_build_powerbi_dataset.py
│   ├── 04_build_annotation_worksheets.py
│   ├── 05_evaluate_models.py
│   ├── 06_descriptive_statistics.py
│   ├── 07_load_database.py
│   ├── 08_build_statistical_report.py
│   ├── 09_build_research_paper.py
│   ├── 10_build_policy_brief.py
│   ├── 11_build_training_worksheets.py
│   └── 12_compare_models.py
├── src/         # reusable Python modules
│   ├── __init__.py
│   ├── absa.py
│   ├── active_learning.py
│   ├── config.py
│   ├── crosstabs.py
│   ├── data_quality.py
│   └── ... 15 more
├── README.md
└── requirements.txt
```

The project folder, shown in the terminal

Folder	What lives there
data/	the original survey file — never modified
db/	the database design and saved analysis queries
src/	21 reusable Python modules
scripts/	12 numbered steps, run in order
notebooks/	a Google Colab notebook for the GPU model run
output/	everything the scripts produce, incl. labelling sheets
results/	model comparison summary (public)
docs/	this report, the presentation and figures
reports/	statistical report, paper draft, policy brief

Folders holding farmers' personal details (data/, output/) stay private and are not uploaded to GitHub.

6. Workflow — How It All Flows

From one Excel file to policy answers, in five stages.

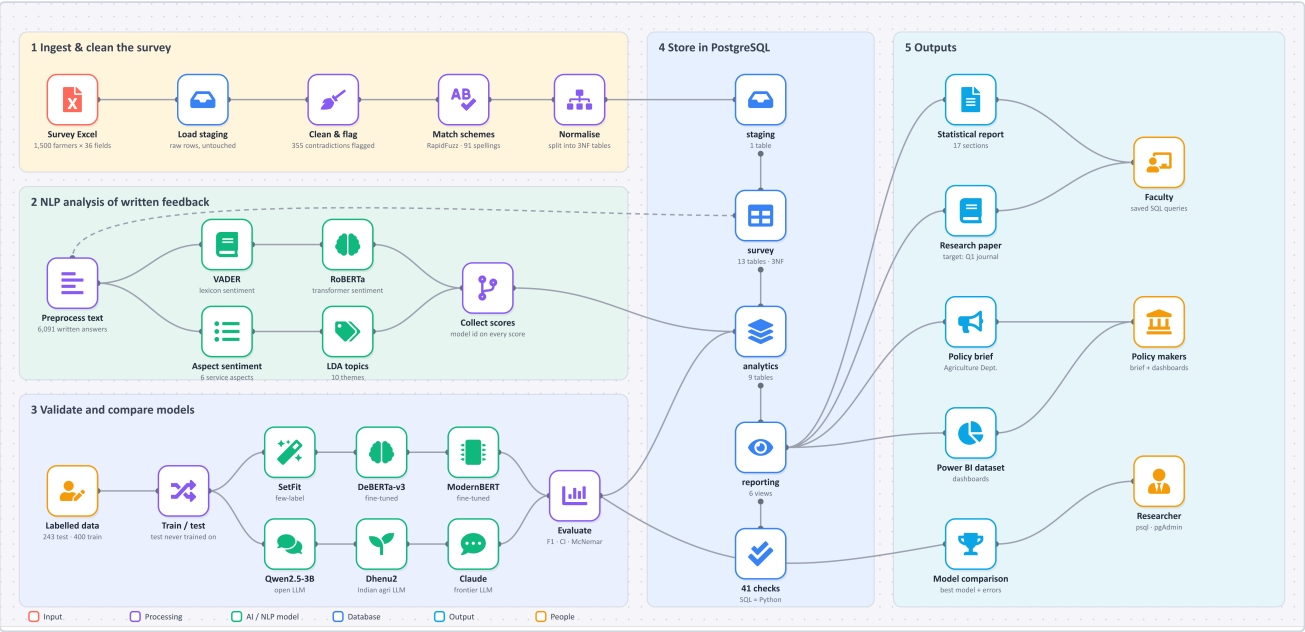


Figure 6.1 — The complete project workflow. Colours show the kind of step: input, processing, AI model, database, output, people.

6.1 The five stages in plain words

Stage	What happens	Result
1 · Ingest and clean	the Excel file is copied untouched, then cleaned: scheme names matched, lists split, contradictions flagged	1,500 clean, traceable records
2 · NLP analysis	four models read the written answers: overall sentiment, sentiment per aspect, and recurring topics	11,151 AI scores
3 · Validate and compare	labelled answers are split into a test set and a training set; eleven models are trained or prompted and scored	a fair ranking of models
4 · Store	everything goes into four layers of the database in one safe step, then 41 automatic checks run	one trusted source of truth
5 · Outputs	reports, policy brief, dashboard data and the paper are all generated from the stored data	answers for each kind of user

6.2 Data management workflow

Two rules guide how data moves: the original file is **never edited**, and human labels are **never overwritten**. Because of this, I can rebuild everything from scratch in about 14 seconds, and get exactly the same answers every time.

7. System Architecture

Users → application → database → output.

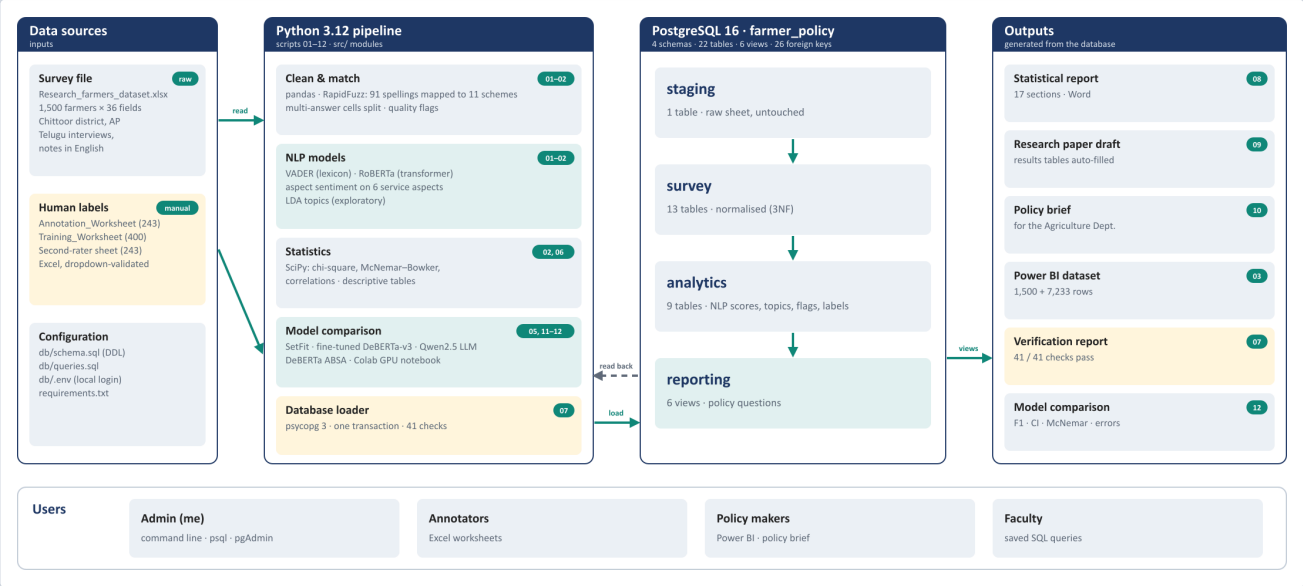
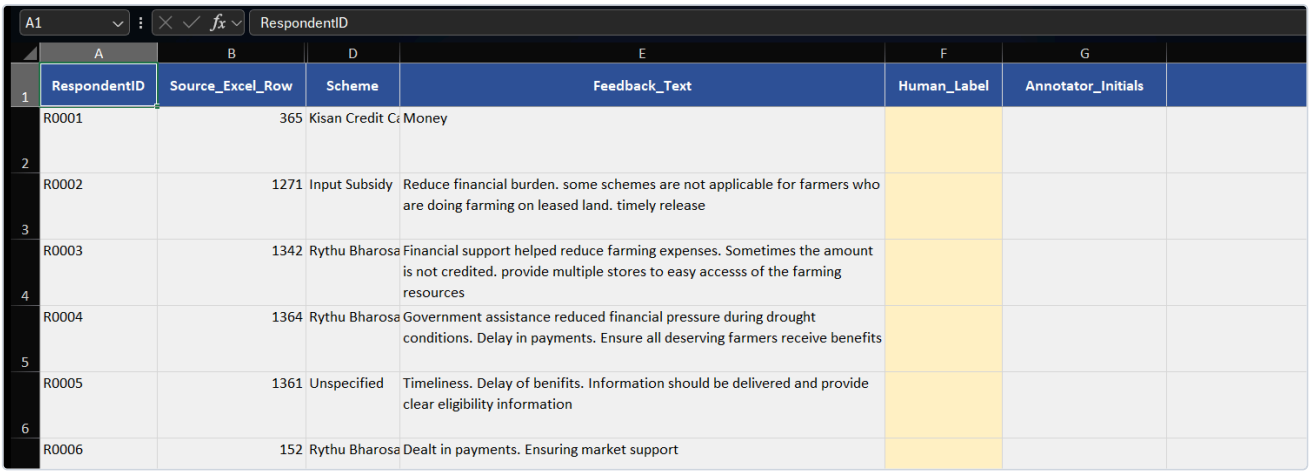


Figure 7.1 — System architecture

On the left are the inputs: the survey file and the human labels. In the middle, the Python application cleans the data, runs the NLP models and statistics, and loads the database in a single transaction — so a failure can never leave half-loaded data behind. The database is organised into four layers, and after every load Python reads the results back and checks that both sides agree. On the right are the outputs, and along the bottom are the four kinds of users and the tool each one uses.

User	How they use the system
Me (researcher)	run the scripts; browse the data in pgAdmin
Annotators	label answers in Excel sheets with one-key shortcuts (1 = Positive, 2 = Neutral, 3 = Negative)
Policy makers	read the policy brief and the Power BI dashboard data
Faculty	check any result against the original survey row



A1 | RespondentID

	A	B	D	E	F	G
1	RespondentID	Source_Excel_Row	Scheme	Feedback_Text	Human_Label	Annotator_Initials
2	R0001	365	Kisan Credit C	Money		
3	R0002	1271	Input Subsidy	Reduce financial burden. some schemes are not applicable for farmers who are doing farming on leased land. timely release		
4	R0003	1342	Rythu Bharosa	Financial support helped reduce farming expenses. Sometimes the amount is not credited. provide multiple stores to easy accesss of the farming resources		
5	R0004	1364	Rythu Bharosa	Government assistance reduced financial pressure during drought conditions. Delay in payments. Ensure all deserving farmers receive benefits		
6	R0005	1361	Unspecified	Timeliness. Delay of benifits. Information should be delivered and provide clear eligibility information		
	R0006	152	Rythu Bharosa	Dealt in payments. Ensuring market support		

Figure 7.2 — The Excel labelling sheet used by annotators (farmer names hidden)

8. How the Data Is Organised

The database design: layers, tables, links and rules.

8.1 Four layers

Layer	Tables	Purpose
staging	1	the raw sheet exactly as received
survey	13	the cleaned survey, split into well-structured tables
analytics	9	AI results, topics, quality flags and human labels
reporting	6 views	ready-made answers to policy questions

8.2 Entity-relationship (ER) diagrams

These diagrams were generated directly from the live database, so they match what was actually built. Yellow tags mark primary keys (each row's unique identity), green tags mark foreign keys (links to another table), and the crow's-foot symbol marks the "many" side of a link.

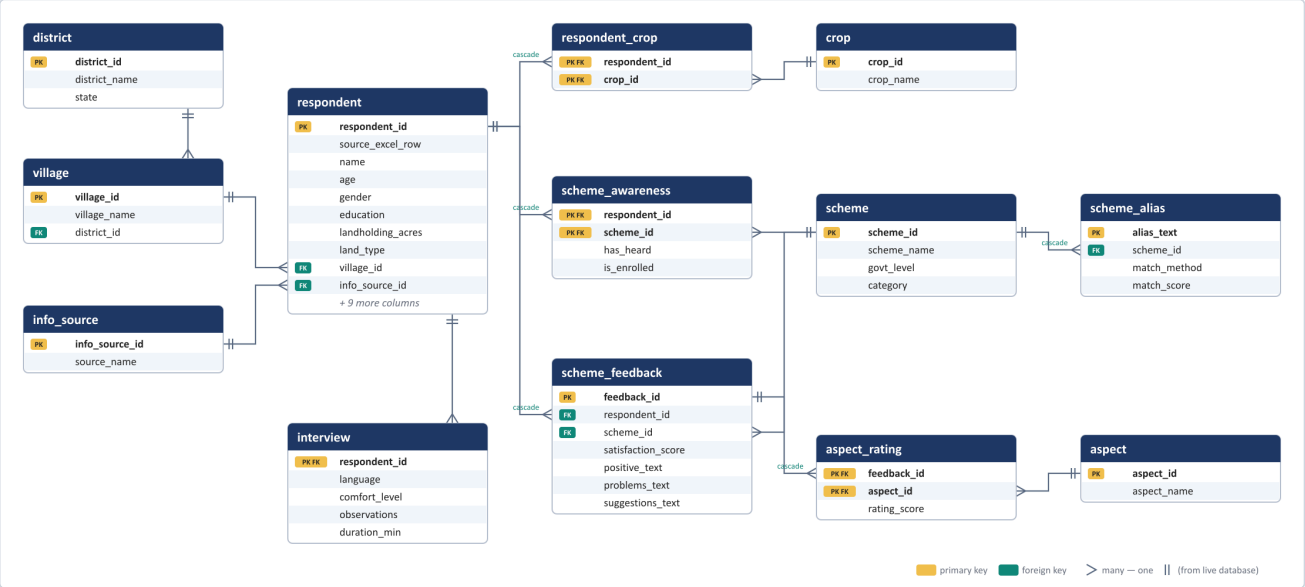


Figure 8.1 — ER diagram of the survey layer: the farmer is at the centre

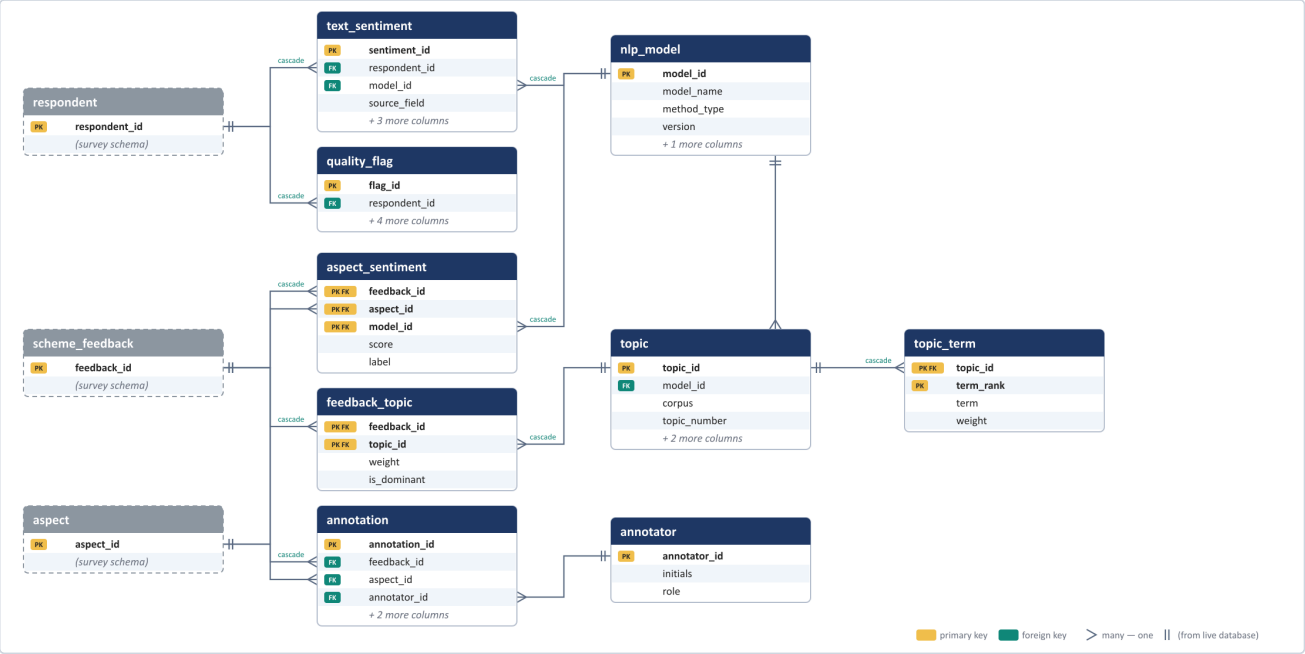


Figure 8.2 — ER diagram of the analytics layer: every AI result points back to the farmer and to the model that produced it

8.3 Normalisation — from one wide row to clean tables

In the original sheet, one row tried to hold everything: lists inside cells, six rating columns side by side, and the district repeated for every farmer in a village. I broke it apart step by step. Lists became separate linking tables (first normal form), and facts that depend on something other than the farmer — like the district of a village — moved into their own tables (second and third normal form). Now every fact is stored exactly once.

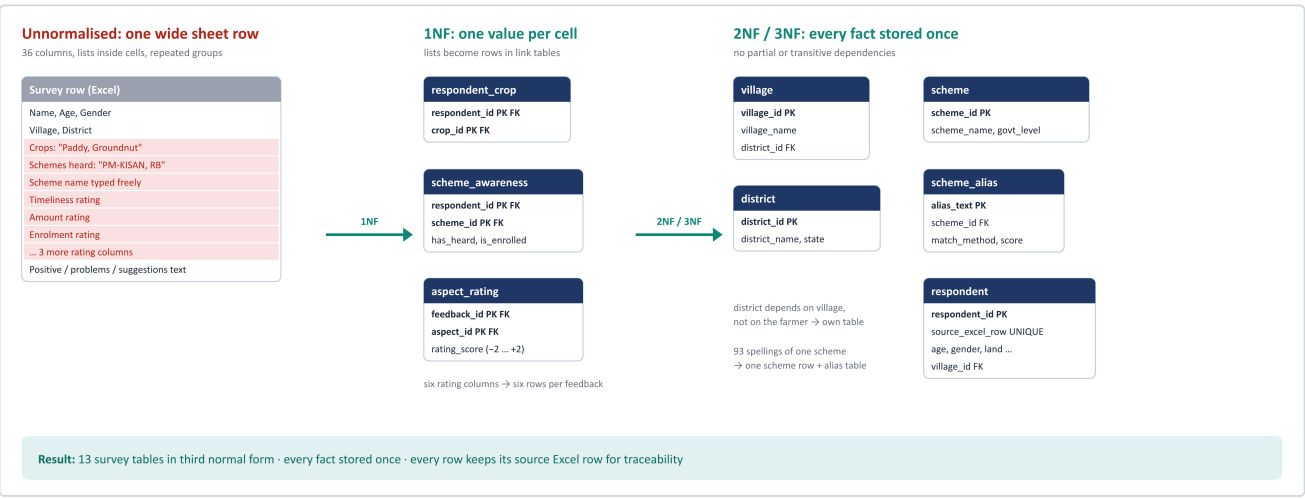
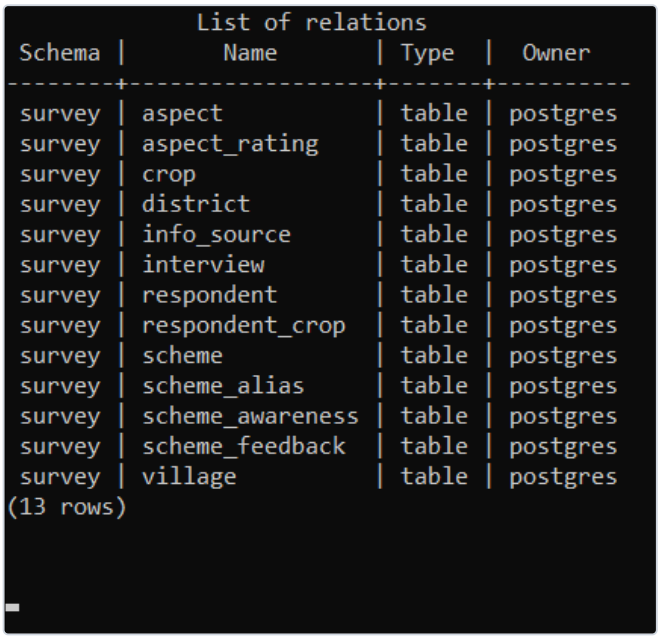


Figure 8.3 — Normalisation from the spreadsheet to third normal form

8.4 Keys, links and rules

Rule type	Count	What it guarantees
Primary keys	23	every row has a unique identity
Foreign keys	26	every link points to something that really exists
Unique rules	13	no duplicates — e.g. one feedback per farmer per scheme
Check rules	25	values stay sensible — ratings between −2 and +2, age 15–100
Indexes	43	fast searching, including full-text search of comments

I also decided what happens when something is deleted. Shared lists like schemes and crops are **protected** — they cannot be deleted while any farmer uses them. A farmer's own records are **removed together** with the farmer (16 such links), so if someone withdraws consent, one deletion removes everything about them.



```
             List of relations
 Schema |       Name       | Type  |  Owner
--------+------------------+-------+----------
 survey | aspect           | table | postgres
 survey | aspect_rating    | table | postgres
 survey | crop             | table | postgres
 survey | district         | table | postgres
 survey | info_source      | table | postgres
 survey | interview        | table | postgres
 survey | respondent       | table | postgres
 survey | respondent_crop  | table | postgres
 survey | scheme           | table | postgres
 survey | scheme_alias     | table | postgres
 survey | scheme_awareness | table | postgres
 survey | scheme_feedback  | table | postgres
 survey | village          | table | postgres
(13 rows)
```

Figure 8.4 — The 13 survey tables inside the running database

9. LLMs, NLP Models and Modules

Eleven ways of reading a farmer's feedback — from a word list to large language models.

9.1 Project modules

Step	Module	What it does
01	Pipeline	cleans the survey, matches scheme names, runs VADER and RoBERTa
02	Analysis	data-quality checks, statistical tests, topic models
03	Dashboard data	builds the Power BI dataset
04	Test sheets	picks a fair 20% sample for human labelling
05	Evaluation	scores the stored models against human labels
06	Statistics	all tables for the reports
07	Database load	builds and fills the database, runs 41 checks
08–10	Reports	statistical report, paper draft, policy brief
11	Training sheets	picks 400 training answers — half where models are unsure, half random
12	Model comparison	trains, prompts and scores all eleven models

9.2 Types of NLP used

NLP type	Model(s)	How it works, simply
Lexicon-based sentiment	VADER + farming word list	adds up the positive and negative words it recognises
Pretrained transformer	RoBERTa (trained on tweets)	a neural network that already knows sentiment, used as-is
Aspect-based sentiment	keyword method · DeBERTa ABSA	finds the mood about each part of a scheme separately — timeliness, amount, enrolment …
Topic modelling	LDA	groups complaints into recurring themes
Few-label learning	SetFit	learns a new task from just a few hundred examples
Fine-tuning	DeBERTa-v3 · ModernBERT	retrains a whole transformer on farmers' answers
LLM prompting	Dhenu2 · Qwen2.5 · Claude	given the labelling guide (and sometimes three examples), the model decides the label

9.3 The large language models

Model	Made by	Size	Why I included it
Dhenu2	KissanAI (India)	3 billion	built for Indian agriculture — does farming knowledge help?
Qwen2.5-3B-Instruct	Alibaba	3 billion	a general LLM of the same size, for a fair comparison
Claude Opus 5.5	Anthropic	frontier	a strong modern LLM as an upper reference

Model	Made by	Size	Why I included it
ModernBERT · DeBERTa-v3	Answer.AI · Microsoft	~150 million	smaller models that can be fully trained on our labels

The 3-billion-parameter LLMs were loaded in 4-bit precision so they fit on my laptop's 4 GB graphics card.

9.4 How I tested the models fairly

Part	Size	Used for
Test set	243 answers	scoring only — no model ever learns from these
Training set	400 answers	teaching the trainable models and choosing the 3 examples for LLMs
Aspect test set	131 answers (274 aspect mentions)	scoring the aspect-level models

Labels were produced jointly by me and one other annotator. For the training set, an AI first suggested labels and we checked every one; we changed 86 of 400 of them. Each trainable model was run with five different random seeds, and I report the average with 95% confidence intervals and a significance test, so lucky runs cannot fool the comparison.

```
PS> python scripts\12_compare_models.py --help
usage: 12_compare_models.py [-h] [--seeds SEEDS] [--epochs EPOCHS] [--setfit-backbone SETFIT_BACKBONE]
                            [--finetune-backbone FINETUNE_BACKBONE] [--llm LLM] [--shots SHOTS] [--llm-4bit] [--abs
                            [--skip-setfit] [--skip-finetune] [--skip-absa] [--no-learning-curve]

Compare sentiment models: zero-shot baselines, models trained on Training_Worksheet.xlsx, and LLMs.

Test set = the human labels in Annotation_Worksheet.xlsx (stratified random 20% sample, never used for training).
Training set = the human labels in Training_Worksheet.xlsx (script 11; disjoint from the test set).

options:
  -h, --help            show this help message and exit
  --seeds SEEDS         training runs per trained model (default: 5)
  --epochs EPOCHS       fine-tuning epochs (default: 4)
  --setfit-backbone SETFIT_BACKBONE
                        SetFit sentence encoder (default: sentence-transformers/all-mpnet-base-v2)
  --finetune-backbone FINETUNE_BACKBONE
                        transformer to fine-tune (default: microsoft/deberta-v3-base)
  --llm LLM             Hugging Face model id; repeatable (needs a GPU in practice) (default: [])
  --shots SHOTS         few-shot examples per class for LLMs (default: 3)
  --llm-4bit            load LLMs in 4-bit (needs bitsandbytes and a CUDA GPU) (default: False)
  --absa-model ABSA_MODEL
                        aspect-sentiment model (default: yangheng/deberta-v3-base-absa-v1.1)
  --skip-setfit
  --skip-finetune
  --skip-absa
  --no-learning-curve
```

Figure 9.1 — The model comparison module and the models it uses by default

10. What the Data Tells Us — Statistics

Who the farmers are, what they know, and how they feel.

51 yrs	2.0 ac	20 yrs	73.7%	90.5%
average age	median land	farming experience	aware of schemes	interviews in Telugu

Gender

	Farmers	Share
Male	989	67.2%
Female	483	32.8%

Land holding

	Farmers	Share
Marginal (<2.5 ac)	581	53.3%
Small (2.5-5 ac)	246	22.5%
Semi-medium (5-10 ac)	136	12.5%
Large (10+ ac)	128	11.7%

Education

	Farmers	Share
High school (9-10)	288	19.6%
Primary school (1-5)	275	18.7%
No formal education	263	17.9%
Intermediate (11-12)	229	15.6%
Graduate	222	15.1%
Middle school (6-8)	125	8.5%

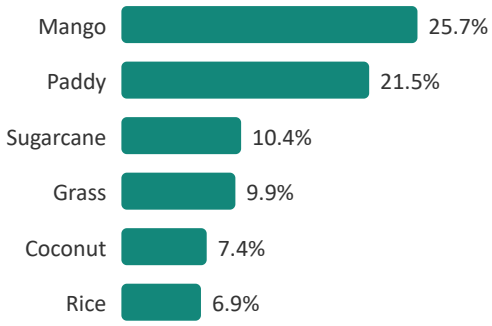


Figure 10.1 — Main crops (% of farmers)

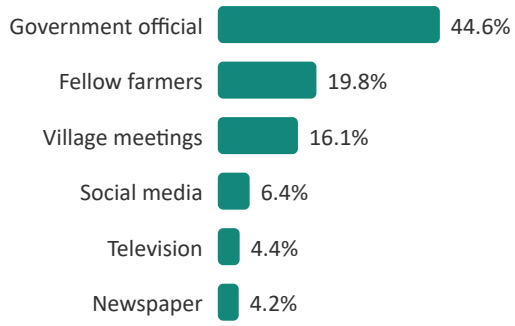


Figure 10.2 — Where farmers hear about schemes (%)

10.1 How farmers feel

Using the first-stage model, 55.6% of written feedback sounded positive, 21.5% neutral and 22.9% negative. As Chapter 11 shows, that model turned out to be unreliable, which is exactly why I tested better ones before drawing conclusions.

What farmers complain about most (topic model)

Theme	Share of complaints	Typical words
Payment delays & bank verification	22%	payment, delayed, bank, delay, eligible, family
Online registration & documents	18%	application, online, registration, document, required, delay
Insufficient support for crop needs	17%	need, insufficient, crop, official, support, requirement
Late credit of benefits	22%	time, delay, sufficient, receiving, late, credited

Theme	Share of complaints	Typical words
Low awareness & slow office approval	21%	time, application, lack, long, difficulty, awareness

What farmers suggest

Rank	Suggestion
1	Increase the benefit amount
2	Simplify / speed up the application process
3	Run more awareness programs
4	Ensure faster, more reliable disbursement

10.2 Does sentiment depend on who the farmer is?

Chi-square tests check whether two things are connected. All of these are statistically significant ($p < 0.05$), but the connections are weak (Cramér's V around 0.1–0.15) — background matters a little, not a lot.

Sentiment compared with	Chi-square	p-value	Cramér's V (strength)
Education	58.53	9.2e-07	0.143
Landholding	32.89	1.1e-05	0.125
Info Source	27.42	3.7e-02	0.107
Scheme	42.32	1.0e-03	0.133
Gender	20.37	3.8e-05	0.119
Income	48.2	5.7e-07	0.13

11. The Main NLP Result

Which model understands farmers best?

All eleven models were scored on the same 243 answers that no model had seen during training. I use **macro-F1** as the main score: it averages how well a model does on positive, neutral and negative feedback, so a model cannot look good just by guessing the most common answer.

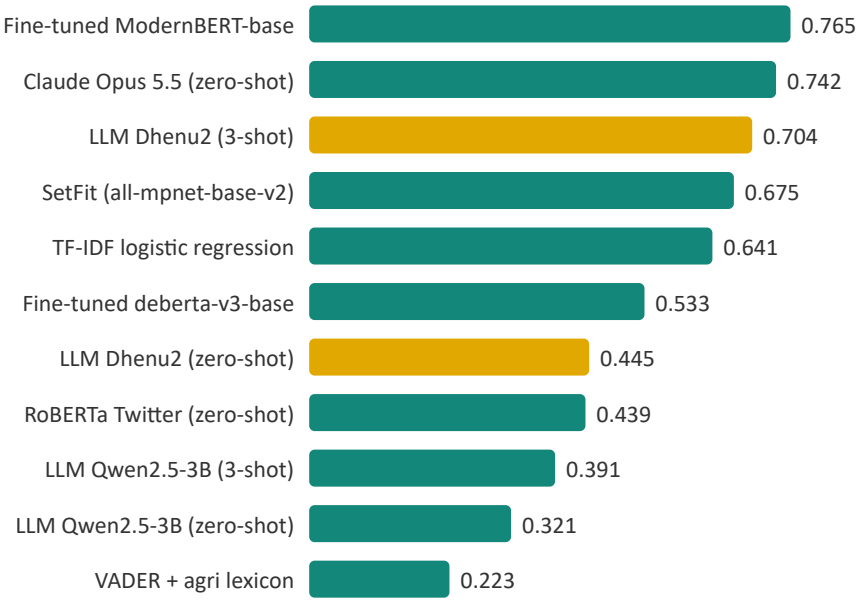


Figure 11.1 — Macro-F1 of every model (higher is better). The Indian-agriculture LLM is highlighted.

Model	Type	Accuracy	Macro-F1	95% confidence range	Average ± spread over 5 runs
Fine-tuned ModernBERT-base	trained (fine-tuned)	0.835	0.765	0.687 - 0.832	0.750 ± 0.034
Claude Opus 5.5 (zero-shot)	LLM (prompted, external)	0.786	0.742	0.676 - 0.801	—
LLM Dhenu2-In-Llama3.2-3B-Instruct (3-shot)	LLM (prompted)	0.749	0.704	0.63 - 0.766	—
SetFit (all-mpnet-base-v2)	trained (few-label)	0.840	0.675	0.592 - 0.752	0.685 ± 0.022
TF-IDF logistic regression	trained baseline	0.745	0.641	0.562 - 0.718	—
Fine-tuned deberta-v3-base	trained (fine-tuned)	0.749	0.533	0.476 - 0.595	0.584 ± 0.080
LLM Dhenu2-In-Llama3.2-3B-Instruct (zero-shot)	LLM (prompted)	0.519	0.445	0.38 - 0.51	—
RoBERTa Twitter (zero-shot)	zero-shot baseline	0.506	0.439	0.367 - 0.512	—
LLM Qwen2.5-3B-Instruct (3-shot)	LLM (prompted)	0.457	0.391	0.317 - 0.458	—
LLM Qwen2.5-3B-Instruct (zero-shot)	LLM (prompted)	0.300	0.321	0.268 - 0.376	—
VADER + agri lexicon	zero-shot baseline	0.235	0.223	0.163 - 0.279	—

> **Winner: Fine-tuned ModernBERT-base** — macro-F1 0.765, accuracy 83.5%. Its lead over Claude Opus 5.5 (zero-shot), SetFit (all-mpnet-base-v2) is not statistically significant, so I treat them as tied at the top; it is significantly better than every other model.

11.1 How the best model does on each kind of feedback

Feedback type	Precision	Recall	F1	Test answers
Positive	0.873	0.724	0.791	76
Neutral	0.833	0.944	0.885	143
Negative	0.722	0.542	0.619	24

It is strongest on neutral and positive feedback. Negative feedback is the hardest — only 24 test answers were negative, so there were fewer examples to learn from.

11.2 How much labelled data is needed?

Training answers	Macro-F1 (TF-IDF model)
100	0.494
200	0.567
300	0.633
400	0.641

Every extra 100 labelled answers improved the score, which tells me that labelling more answers would make the models even better.

11.3 Sentiment about each aspect of a scheme

Method	Aspect mentions found	Accuracy	Macro-F1
Keyword ABSA + VADER (current pipeline)	201 of 274	0.343	0.373
DeBERTa ABSA (deberta-v3-base-absa-v1.1)	274 of 274	0.522	0.433

The purpose-built DeBERTa aspect model handled every aspect mention and scored higher than my original keyword method.

12. Research Findings

What this project adds to what we know.

Finding 1 — Knowing about a scheme is not the same as joining it

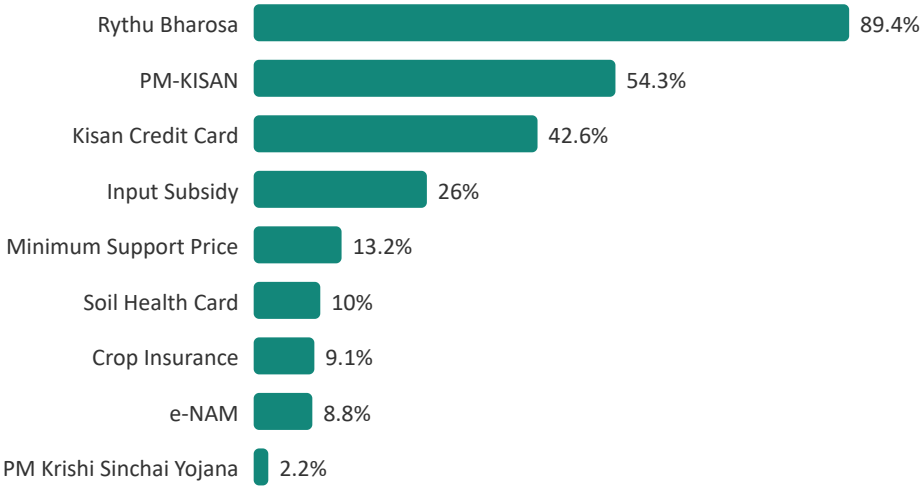


Figure 12.1 — Enrolled, among farmers who know the scheme (%)

89.4% of farmers who know Rythu Bharosa are enrolled, but only 9.1% of the 143 farmers who know Crop Insurance are. For crop insurance the problem is clearly not awareness — it is getting farmers through the door.

Finding 2 — Ratings and farmers' own words often disagree

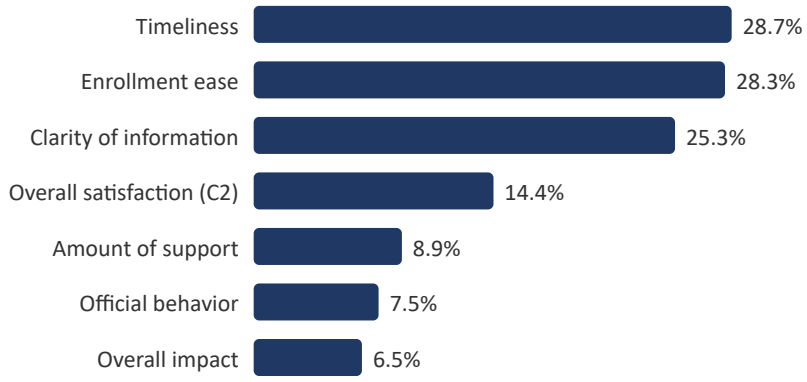


Figure 12.2 — How often a farmer's rating contradicts their own comment (%)

The disagreement is highest for timeliness (28.7%) and enrolment (28.3%), and a McNemar–Bowker test shows it is far beyond chance ($\chi^2 = 192.73$, $p < 0.001$). In other words, satisfaction scores alone hide real problems.

Finding 3 — Enrolment is the weakest aspect of every major scheme

Scheme	Weakest aspect	Negative mentions
Input Subsidy	Enrolment	53.8%
Kisan Credit Card	Enrolment	50.0%
PM-KISAN	Enrolment	47.6%
Rythu Bharosa	Enrolment	40.6%

All four major schemes share the same weak point, which points to one common registration process that needs fixing, rather than four separate problems.

Finding 4 — Popular sentiment tools do not understand farmer feedback

VADER, a word-list tool still used in many studies, scored only 0.22, and RoBERTa trained on tweets 0.44. Models that learned from farmers' answers reached up to 0.77. Any study that applies these tools to farmer surveys without testing them first may be drawing the wrong conclusions.

Finding 5 — Agricultural knowledge helps LLMs

With just three examples, the Indian-agriculture LLM **Dhenu2 scored 0.70**, while a general LLM of exactly the same size, **Qwen2.5-3B, scored 0.39**. Domain knowledge clearly helps a model understand what farmers mean.

> Findings 1–3 come straight from the survey answers. Findings 4–5 come from the model comparison. Together they form the core of the research paper.

13. Testing and Performance

How I made sure the system can be trusted.

13.1 The database checks itself

After every load, 41 automatic checks compare what the database says with what the Python analysis calculated — row counts, sentiment totals, disagreement rates, scheme figures and awareness counts. All 41 pass.

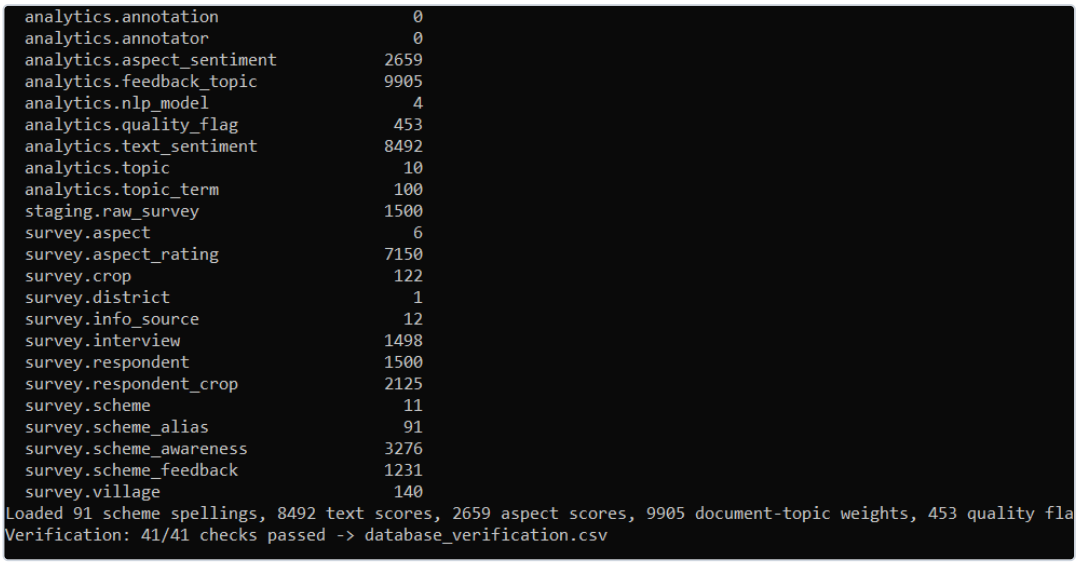


Figure 13.1 — Loading the database: row counts and "Verification: 41/41 checks passed"

13.2 Trying to break the rules on purpose

What I tried to insert	Rule that stopped it	Result
a rating of 5 (allowed: −2 to +2)	check rule	rejected
feedback from a farmer who does not exist	foreign key	rejected
two farmers with the same ID	primary key	rejected
a sentiment label of "Happy"	check rule	rejected

Answers that contradict each other but are genuine — like a farmer who says they know no schemes but then rates one — are not rejected. They are kept and flagged (453 flags), because deleting real answers would bias the results.

13.3 Performance

13.7 s	< 15 ms	16 MB	~33 s
full rebuild from Excel	every policy question	database size	to fine-tune a model on GPU

14. Challenges and How I Solved Them

Most of the hard parts came from real, messy data.

Type	Challenge	How I solved it
Data handling	the same scheme typed 93 different ways	fuzzy matching, plus a table recording every spelling and how it was matched
Data handling	several answers packed into one cell	split them using the questionnaire's list of options, into separate linking tables
Data handling	355 farmers gave contradictory answers	kept the answers and flagged them, instead of deleting
Data handling	"no problems" was being read as negative	treated such empty-meaning answers as empty
Database design	six rating columns side by side	turned them into one row per feedback and aspect
Database design	results from many models in one place	a models table — every score carries the ID of its model
Database design	keeping the database and analysis in sync	41 automatic checks after every load
Technical	LLMs too big for my laptop	4-bit quantisation on the GPU
Technical	GPU memory filling up between models	freeing memory after each model and caching results so runs can resume
Method	interviews in Telugu, notes in English	used English models and noted it as a limitation
Method	topic models changed between runs	reported them as exploratory only

15. Conclusion, Links and Future Scope

Where the project stands, and where it goes next.

15.1 What I achieved

- A well-structured database of 22 linked tables holding 40,286 records, with 26 links and 25 rules that keep the data honest — rebuilt and verified automatically.
- An NLP layer where every result is stored with the model that produced it, and a fair comparison of 11 models, including an Indian-agriculture LLM.
- Clear policy answers: the awareness–enrolment gap, the hidden disagreement between ratings and words, and the shared enrolment weakness.
- Reports, a policy brief, dashboard data and a research paper draft — all generated from the data.

15.2 What I learned

- How to turn messy real-world data into a clean, normalised design.
- How keys, rules and transactions protect data quality in practice.
- How to connect Python to a database safely.
- How to evaluate AI models properly — held-out test sets, repeated runs, confidence intervals and significance tests.
- How to run large language models efficiently on modest hardware.

15.3 Limitations

- One district and one survey round.
- Interviews were in Telugu but recorded in English, so some nuance may be lost.
- Labels were made jointly as one agreed set, so agreement between two independent labellers could not be measured.
- Negative feedback was rare in the test set (24 answers), so scores on it are less certain.

15.4 Future scope

> **Research paper — in progress.** I am now writing up these findings as a research paper **targeting a Q1 journal**, under the guidance of Prof. Sangeetha S. Its core contributions are the gap between what farmers rate and what they write, the awareness–enrolment gap, and a validated comparison of NLP models including an Indian-agriculture LLM.

- Label more answers, with two independent labellers, to strengthen the evaluation.
- Use the best model to recompute the aspect-level findings for the paper.
- Connect a live Power BI dashboard directly to the database.
- Extend to more districts and survey rounds, and host the database online.
- Test models that read Telugu directly.

15.5 Links

Resource	Link
GitHub repository — code, database design, results	https://github.com/singhkrish2006/farmer-scheme-policy-intelligence
Project presentation (PPT)	https://github.com/singhkrish2006/farmer-scheme-policy-intelligence/blob/main/docs/Project_Demonstration.pptx
This report (PDF)	https://github.com/singhkrish2006/farmer-scheme-policy-intelligence/blob/main/docs/Complete_Project_Report.pdf

The raw survey and the labelling sheets are not published because they contain farmers' personal details.

Appendix

Running the project, and credits.

A. How to run the project

Step	Command	What it does
1	pip install -r requirements.txt	installs everything
2	python scripts/01_run_pipeline.py	cleans the survey and runs the first NLP models
3	python scripts/02_run_analysis.py	quality checks, statistics, topics
4	python scripts/07_load_database.py	builds the database and runs the 41 checks
5	python scripts/05_evaluate_models.py	scores the stored models against human labels
6	python scripts/12_compare_models.py	the full eleven-model comparison

B. Photo credits

Photo	Author	Licence	Source
RICE FARMS,SN KANDRIGA CHITTOOR DIST AP - panoramio.jpg	rajaraman sundaram	CC BY 3.0	Wikimedia Commons
Farmer ploughing with cattle, Nandyal District, Andhra Pradesh.jpg	PJeganathan	CC BY-SA 3.0	Wikimedia Commons
Gudimallam village farms and environment, Andhra Pradesh India.jpg	Ms Sarah Welch	CC0	Wikimedia Commons
Nunna Rythu Bharosa Kendram.jpg	Veera Ram Kumar	CC0	Wikimedia Commons